实 验 一　汇 编 语 言 源 程 序 的 输 入

一、实验目的

1.通过实验了解和熟悉微机系统的配置。

2.学习在 DEBUG 状态下输入汇编源程序的方法。

3.初步掌握调试(在 DEBUG 状态下)的过程。

二、实验原理

1. 本实验要求在 DEBUG 状态下输入汇编源程序,并用 DEBUG 命令进行调试。用单步跟踪的方法验证指令的功能。

2. 以下是给定的参考程序,并在实验时在每条指令的“；　”符号右边按要求填写指令的执行结果。

注:⑴　微机进入 DEBUG 状态下之后,一切立即数和地址数据均被默认为十六进制数,在输入时数的后面不加后缀“H”;

⑵　在 DEBUG 状态下执行程序时,“INT 20H”指令可使系统执行完该指令前的程序时返回到“-”提示符状态，并且恢复 CS 和 IP 寄存器原来的值。

三、实验步骤

1. 开机后进入 DOS 系统，

C＞　DEBUG↙　(↙回车符)

－ (为 DEBUG 提示符)

当显示器出现提示符“－”时，说明已进入 DEBUG 状态,这时,可用 DEBUG 命令进行操作。

2. 用 DEBUG 的 Register 命令检查所有寄存器内容,并作记录。命令格式：

R [寄存器名]

该命令的功能是显示寄存器的内容，或修改某一指定寄存器内容，若[寄存器名]缺省，则显示所有寄存器内容。例如：

－R

3. 用 DEBUG 的 Assemble 命令输入汇编源程序。格式：

A [内存地址]

注：用“[ ]”符号括起来的部分表示可以省略。

该命令的功能是从指定的内存地址开始（括号不要输入）逐条输入汇编语言源程序并

汇编成机器码存入内存。若地址缺省，则接上一个 A 命令最后一条指令之后输入汇编语句,若没有用过 A 命令，则从 CS：0100H 地址开始输入。例如：

－A

0CD3：0100－

在输入 A 命令之后,或每输入一条指令之后，显示器的左端给出了内存的段地址和偏移地址。

每条指令均用回车(↙)结束。若输入的指令有语法错误，DEBUG 拒绝接收,并给出提示,此时可以重新输入。程序的最后一条指令输入完之后，再按一次回车键(↙),即可结束汇编命令，回到 DEBUG 提示符“－”状态。

4. 用 DEBUG 的 Unassemble 命令反汇编。命令格式：

U [起始地址[终止地址]]

该命令的功能是从起始地址到终止地址反汇编目标码,缺省值是接上一个 U 命令或从 CS：0100H 地址开始。例如：

－U

显示器上将显示程序的内存地址、指令机器码的汇编源程序三列对照清单。

5. 用 DEBUG 的 Trace 命令单步跟踪程序。命令格式：

T [=起始地址] [指令条数]

该命令的功能是从指定的起始地址开始逐条执行指令,每执行完一条指令,屏幕显示所有寄存器内容和下一条指令地址和指令。若[=起始地址]缺省,则 T 命令从 CS：IP 地址开始执行指令。

例如：

－T↙

重复这一过程,即可看到每条指令执行后,所有寄存器和标志寄存器的标志位内容。此时，要检查内存单元的数据，可用 DEBUG 的 D 命令。

6. 用 DEBUG 的 Dump 命令显示存贮器单元的内容。命令格式：

D[起始地址[终止地址]]

该命令的功能是从起始地址到终止地址,连续显示存贮器单元的内容。若地址缺省，则接上一个 D 命令或从 DS：0100H 地址开始显示。例如：

－D↙

参考程序：

```
MOV    AX, 2000                ; AL=00H
MOV    DS, AX                  ; DS=2000H
NOT    AX                      ; AX=DFFFH
XOR    AX, AX                  ; AX=0000H
DEC    AX                      ; AX=FFFFH
INC    AX                      ; AX=0000H
MOV    BX, 2030                ; BH=20H
MOV    SI, BX                  ; SI=2030H
MOV    [SI], BL                ; [2030H]=30H
MOV    WORD PTR[SI], 10F       ; [2030H]=0FH          [2031H]=01H
MOV    DI, SI                  ; DI=2030H
MOV    [DI+50], BH             ; [DI+50H]=20H
MOV    BH, [SI]                ; BH=0FH
MOV    BL, [DI+50]             ; BL=20H
MOV    SP, 5000
PUSH   AX                      ; AX=0000H   [SS：4FFEH]=00H   [SS：4FFFH]=00H
PUSH   BX                      ; BX=0F20H   [SS：4FFCH]=20H   [SS：4FFDH]=0FH
POP    AX                      ; AX=0F20H
POPF                           ; F=0000H
NEG    BX                      ; BX=F0E0H
XCHG   BX, AX                  ; BX=0F20H
STD                            ; F=0400H
STI                            ; F=0600H
CLD                            ; F=0200H
CLI                            ; F=0000H
ADC    DI, 2050                ; DI=4081H             F=0004H
ADC    SP, DI                  ; SP=9081H             F=0884H
ADC    AX, 1500                ; AX=05E0H             F=0001H
```

```
SUB     AX, BX                  ; AX=F6C0H          BX=0F20H
SHL     AH, 1                   ; AH=ECH
RCL     AX, 1                   ; AX=D981H
SHR     BH, 1                   ; BH=07H
RCR     BL, 1                   ; BL=90H
MOV     CL, 4
MOV     DX, 80F0
ROL     DX, CL                  ; DX=0F08H          CL=04H
INT     20                      ; CS=F000H          IP=1480H
```

四、实验报告要求

1. 整理每条指令执行的结果,填到打印清单的右半部分(应注意内存数据检查的正确性)。

已填写上述代码相应位置。

2. 比较实验记录与理论分析的结果是否相同,若有不同,找出差别及问题所在。

实验记录与理论分析的结果相同。

3．总结本次实验的体会。

通过本次实验，我掌握了如何在 DOSBox 这个 DOS 系统的虚拟环境中操作，包括如何进入 DEBUG 状态并编写代码。此外，我熟悉了汇编语言中每条指令的具体功能，并初步了解了在编写汇编程序时如何使用相应的指令。

在实验过程中，首先通过 DEBUG 的 R 命令查看了寄存器的初始状态，随后使用 A 命令在指定内存地址处输入了给定的汇编源程序，并利用 U 命令对输入的程序进行了反汇编，确认了机器码与汇编指令的对应关系，并使用 T 命令进行单步跟踪，逐步观察每一条指令执行后寄存器内容、内存数据以及标志位的变化情况，进行了实验记录。

标志名	标志为 1	标志为 0
OF 溢出（是/否）	OV	NV
DF 方向（减量/增量）	DN	UP
IF 中断（允许/关闭）	EI	DI
SF 符号（负/正）	NG	PL
ZF 零（是/否）	ZR	NZ
AF 辅助进位（是/否）	AC	NA
PF 奇偶（偶/奇）	PE	PO
CF 进位（是/否）	CY	NC

实验二　数据的建立与传送程序

一、实验目的

1. 继续学习 DEBUG 命令。

2. 验证指令的功能。

二、实验内容

在 DEBUG 状态下，分别输入下面各程序段,每输入完一个程序段,用 G 命令进行连续方式执行程序，在连续执行时，要记录程序的执行结果。

参考程序：

1. 在内存 10000H 单元开始，建立 00H～0FH～00H 31 个数，要求 00H～0FH 数据逐渐增大,0FH～00H 逐渐减小。该程序从内存 CS:0100H 地址开始输入。

```
        MOV     AX, 1000H
        MOV     DS, AX
        MOV     SI, 0
        MOV     CL, 0FH
        XOR     AX, AX
PPE1:   MOV     [SI], AL
        INC     SI
        INC     AL
        DEC     CL
```

```
        JNZ     PPE1
        MOV     CX, 10H
PPE2：  MOV     [SI], AL
        INC     SI
        DEC     AL
        LOOP    PPE2
        INT     20H
```

注：转移指令的符号地址直接用绝对偏移地址，该地址在用 A 命令汇编输入时，可以看到程序全部运行完之后，可用 DEBUG 的 Dump 命令查看建立的数据块内容。例如：

－D1000：00　1E

2. 把上一个程序的执行结果(建立的 31 个字节数据块,其首地址在 10000H)，分几种方式传送到以下指定的区域。

(a) 该程序从内存 CS:0150H 开始输入。把数据块传送到 15050H 开始的存贮区域中。

参考程序：

```
        MOV     AX, 1000H
        MOV     DS, AX
        MOV     SI, 0
        MOV     DI, 5050H
        MOV     CX, 1FH          ; 数据块长度是 31
PPEA：  MOV     AL, [SI]
        MOV     [DI], AL
        INC     SI
        INC     DI
        LOOP    PPEA
        INT     20H
```

检查内存数据块的传送情况，可用“D”命令。

(b) 用串传送指令 MOVSB，把数据块传送到 15150H 开始的区域，该程序从内存 CS:0200H 开始输入。

检查程序最后的执行结果，可用“D”命令，例如：

－D1000：5150↙

(c) 用重复串操作指令“REP　MOVSB”把数据块传送到 15250H 开始的区域。该程序从 CS：250H 地址开始输入。

检查程序的最后执行结果时，可用：

－D1000: 5250H

(d) 用串操作的减量工作方式，把数据块传送到 25050H 开始的区域。该程序从 CS：0300H 开始输入。

检查程序的最后执行结果，用 D 命令：

－D2000：5050↙

三、实验报告要求

1. 整理每个程序段在实验时的记录内容。

(1)在内存 10000H 单元开始，建立 00H～0FH～00H 共 31 个数

00H～0FH 数据逐渐增大；0FH～00H 数据逐渐减小；程序从 CS:0100H 地址输入。

```
C:\>debug
-a0100
073F:0100 mov ax,1000
073F:0103 mov ds,ax
073F:0105 mov si,0
073F:0108 mov cl,0f
073F:010A xor ax,ax
073F:010C mov [si],al
073F:010E inc si
073F:010F inc al
073F:0111 dec cl
073F:0113 jnz 010c
073F:0115 mov cx,10
073F:0118 mov [si],al
073F:011A inc si
073F:011B dec al
073F:011D loop 0118
073F:011F int 20
073F:0121
-g=0100 011f

AX=00FF  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=001F  DI=0000
DS=1000  ES=073F  SS=073F  CS=073F  IP=011F   NV UP EI NG NZ AC PE NC
073F:011F CD20          INT     20
-d1000:00 1E
1000:0000  00 01 02 03 04 05 06 07-08 09 0A 0B 0C 0D 0E 0F   ........
1000:0010  0E 0D 0C 0B 0A 09 08 07-06 05 04 03 02 01 00      ........
```

（2）把上一个程序的执行结果按不同方式传送到指定区域

上一个程序建立的 31 个字节数据块首地址为 10000H（段地址 1000H，偏移地址 0000H）。下面分别用四种方式实现数据块传送。

(a) 普通循环传送到 15050H

程序从 CS:0150H 开始输入，把数据块传送到 15050H 开始的存储区域。

```
073F:011F CD20          INT     20
-d1000:00 1E
1000:0000  00 01 02 03 04 05 06 07-08 09 0A 0B 0C 0D 0E 0F   ........
1000:0010  0E 0D 0C 0B 0A 09 08 07-06 05 04 03 02 01 00      ........
-a0150
073F:0150 mov ax,1000
073F:0153 mov ds,ax
073F:0155 mov si,0
073F:0158 mov di,5050
073F:015B mov cx,1F
073F:015E mov al,[si]
073F:0160 mov [di],al
073F:0162 inc si
073F:0163 inc cdi
              ^ Error
073F:0163 inc di
073F:0164 loop 015e
073F:0166 int 20
073F:0168
-g=0150 0166

AX=1000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=001F  DI=506
DS=1000  ES=073F  SS=073F  CS=073F  IP=0166   NV UP EI PL NZ NA PE NC
073F:0166 CD20          INT     20
-d1000:5050
1000:5050  00 01 02 03 04 05 06 07-08 09 0A 0B 0C 0D 0E 0F   ........
1000:5060  0E 0D 0C 0B 0A 09 08 07-06 05 04 03 02 01 00 00   ........
1000:5070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ........
1000:5080  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ........
1000:5090  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ........
1000:50A0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ........
1000:50B0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ........
1000:50C0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ........
```

(b) 用 MOVSB 串传送到 15150H

程序从 CS:0200H 开始输入串传送指令 MOVSB 把数据块传送到 15150H 区域。

代码：

```
        MOV     AX, 1000H       ; 源数据段地址 1000H
```

```
MOV     DS, AX
MOV     ES, AX

MOV     SI, 0           ; 源数据偏移地址 0（10000H）
MOV     DI,5150H
MOV     CX,1FH
CLD
REP     MOVSB
INT     20H             ; 程序结束
```

```
-a0200
073F:0200 mov ax,1000
073F:0203 mov ds,ax
073F:0205 mov es,ax
073F:0207 mov si,0
073F:020A mov di,5150
073F:020D mov cx,1f
073F:0210 cld
073F:0211 rep movsb
073F:0213 int 20
073F:0215
-g=0200 0213

AX=1000  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=001F  DI=516F
DS=1000  ES=1000  SS=073F  CS=073F  IP=0213   NV UP EI PL NZ NA PO NC
073F:0213 CD20          INT     20
-
-d1000:5150
1000:5150  00 01 02 03 04 05 06 07-08 09 0A 0B 0C 0D 0E 0F   ..........
1000:5160  0E 0D 0C 0B 0A 09 08 07-06 05 04 03 02 01 00 00   ..........
1000:5170  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ..........
1000:5180  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ..........
1000:5190  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ..........
1000:51A0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ..........
1000:51B0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ..........
1000:51C0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ..........
```

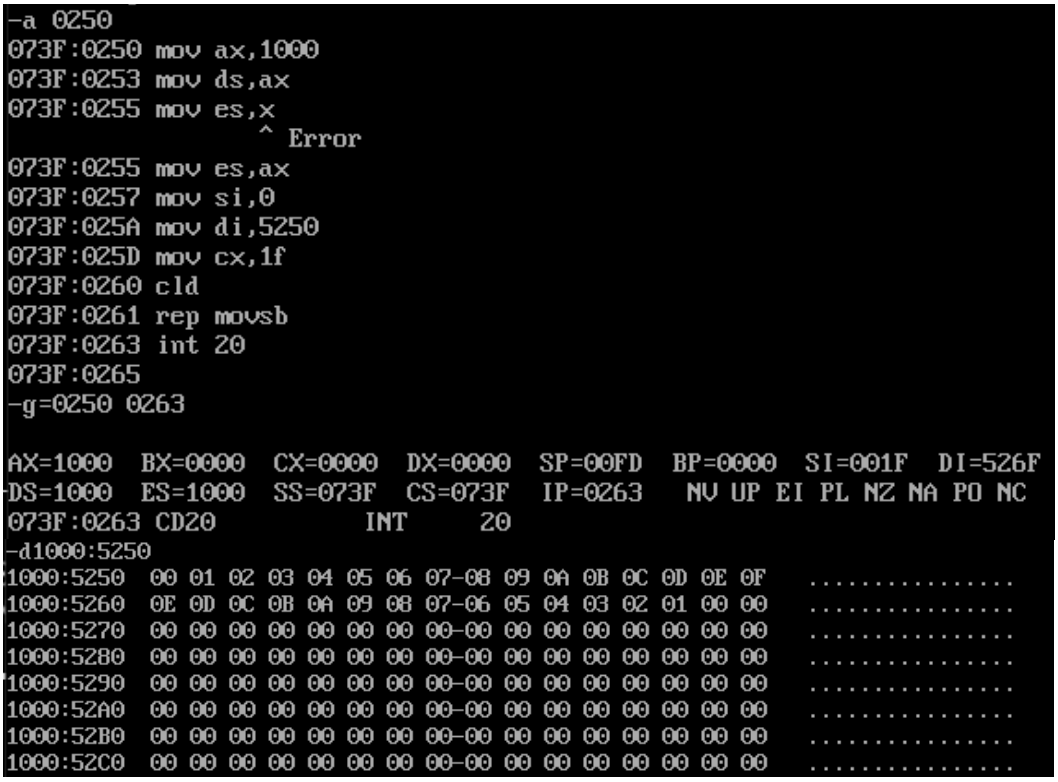
(c) 用 REP MOVSB 传送到 15250H

程序从 CS:0250H 开始输入，用 REP MOVSB 把数据块传送到 15250H 开始的区域。

代码：

```
MOV     AX, 1000H       ; 源数据段地址 1000H
MOV     DS, AX
```

```
MOV        ES,AX
MOV        SI, 0
MOV        DI,5250H
MOV        CX, 1FH            ; 数据长度 31
CLD                           ; 方向标志 DF=0（地址递增）
REP        MOVSB              ; 重复传送直到 CX=0
INT        20H                ; 程序结束
```



(d) 串操作减量方式传送到 25050H

程序从 CS:0300H 开始输入，用减量方式的串传送，把数据块传送到 25050H 开始的区域。

代码：

```
MOV        AX, 1000H          ; 源数据段地址 1000H
MOV        DS, AX
```

```
MOV     SI, 1EH          ; 源数据末地址偏移（10000H+30=1001EH）
MOV     AX, 2500H        ; 目标段地址 2500H（25050H = 2500H:050H）
MOV     ES, AX
MOV     DI, 050H + 1EH   ; 目标末地址偏移（25050H+30=2506EH）
MOV     CX, 1FH          ; 数据长度 31
STD                      ; 方向标志 DF=1（地址递减）
REP     MOVSB            ; 重复传送直到 CX=0
INT     20H              ; 程序结束
```

```
C:\>debug
-a0300
073F:0300 mov ax,1000
073F:0303 mov ds,ax
073F:0305 mov si,1e
073F:0308 mov ax,2500
073F:030B mov es,ax
073F:030D mov di,050+1e
073F:0310 mov cx,1f
073F:0313 std
073F:0314 rep movsb
073F:0316 int 20
073F:0318
-g=0300 0316

AX=2500  BX=0000  CX=0000  DX=0000  SP=00FD  BP=0000  SI=FFFF  DI=004F
DS=1000  ES=2500  SS=073F  CS=073F  IP=0316   NV DN EI PL NZ NA PO NC
073F:0316 CD20          INT     20
-d2000:5050
2000:5050  00 01 02 03 04 05 06 07-08 09 0A 0B 0C 0D 0E 0F   ................
2000:5060  0E 0D 0C 0B 0A 09 08 07-06 05 04 03 02 01 00 00   ................
2000:5070  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
2000:5080  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
2000:5090  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
2000:50A0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
2000:50B0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
2000:50C0  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00   ................
```

2. 比较每个程序段的特点。

A 程序段使用最基本的循环结构，将位于 DS:SI 的数据块逐字节传送至 DS:DI。该过程循环执行 31 次，每次传送一个字节。

B 程序段采用串传送指令 MOVSB，该指令需要一个额外的段寄存器 ES。MOVSB 指令用于传送一个字节，即将 DS:SI 的数据块传送至 DS:DI，并在完成后自动增加 SI 和

DI 的值，从而将传送操作合并为一步。

C 程序段在 B 的基础上进一步简化，使用 REP MOVSB 指令。REP MOVSB 不仅执行字节传送的功能，还包含循环的功能，自动在数据未传送完毕时继续执行。这一程序段采用从小地址到大地址的顺序进行数据传输。

D 程序段与 C 相同，但传送顺序相反，先传送高地址后传送低地址。它使用 STD 指令来表示串操作的减量传送。

3. 分析本次实验出现的问题，找出问题所在。

在使用串操作的减量工作方式（D 程序段）时，需要通过 SI 和 DI 寄存器在每次传送后自动减量。但实验中错误地将 DI 设置为目标区域的首地址，而实际上在减量模式下，DI 应该指向目标区域的末地址（首地址 + 数据长度 - 1）。这导致数据从高地址向低地址传送时，覆盖了预期存储区域之外的内存空间，出现数据存放位置错误。

实验三　分支程序设计

一、实验目的

1.练习分支程序的编写方法。

2.练习汇编语言程序的上机过程。

二、实验原理

1.通过分支程序设计调试和运行，进一步熟悉掌握汇编程序执行的软件环境。

2.通过分支程序的执行过程，熟悉 EDIT 的使用，建立 OBJ 文件 EXE 文件的方法。

三、实验内容

1.将一个字符串变量 string 中的小写字母转换成大写字母并显示出来。

源代码：

```
DATAS SEGMENT
    STRING DB "hello world"
    LEN EQU $-string
DATAS ENDS
CODES SEGMENT
```

```
        ASSUME CS:CODES,DS:DATAS
    START:
        MOV AX,DATAS
        MOV DS,AX
        MOV CX,LEN
        MOV SI,OFFSET STRING
    LOOP1:
        MOV AL,[SI]
        CMP AL,'a'
        JB LOOP2
        CMP AL,'z'
        JA LOOP2
        SUB AL,32
    LOOP2:
        MOV DL,AL
        MOV AH,02H
        INT 21H
        INC SI
        LOOP LOOP1

        MOV DL, 0Ah
        MOV AH, 02h
        INT 21h
        MOV AH,4CH
        INT 21H
    CODES ENDS
        END START
```

运行结果：



2. 给出三个有符号数，编写一个比较相等关系的程序：

（1） 如果这三个数都不相等，则显示 0；

（2） 如果这三个数中有两个数相等，则显示 1；

（3） 如果这三个数都相等，则显示 2；

源代码：

```
DATAS SEGMENT
    NUM1 DW 10
    NUM2 DW 20
    NUM3 DW 10
    COUNT DB 0
DATAS ENDS
CODES SEGMENT
    ASSUME CS:CODES,DS:DATAS
START:
    MOV AX,DATAS
    MOV DS,AX
    MOV AX,NUM1
    MOV BX,NUM2
    MOV CX,NUM3
    CMP AX,BX
    JNE LOOP1
    INC COUNT
LOOP1:
    CMP AX,CX
    JNE LOOP2
```

```
        INC COUNT
LOOP2:
        CMP BX,CX
        JNE LOOP3
        INC COUNT
LOOP3:
        CMP COUNT,2
        JE LOOP4
        CMP COUNT,1
        JE LOOP5
        CMP COUNT,0
        JE LOOP6
LOOP4:
        MOV DL,'2'
        JMP LAST
LOOP5:
        MOV DL,'1'
        JMP LAST
LOOP6:
        MOV DL,'0'
        JMP LAST
LAST:
        MOV AH,02H
        INT 21H
        MOV DL,0AH
        MOV AH,02H
        INT 21H
        MOV AH,4CH
        INT 21H
```

CODES ENDS

END START

运行结果：（给定 NUM1 DW 10、NUM2 DW 20、NUM3 DW 10）



四、实验报告要求

实验报告中要有源程序和执行结果。

实验四　统计学生成绩程序

一、实验目的

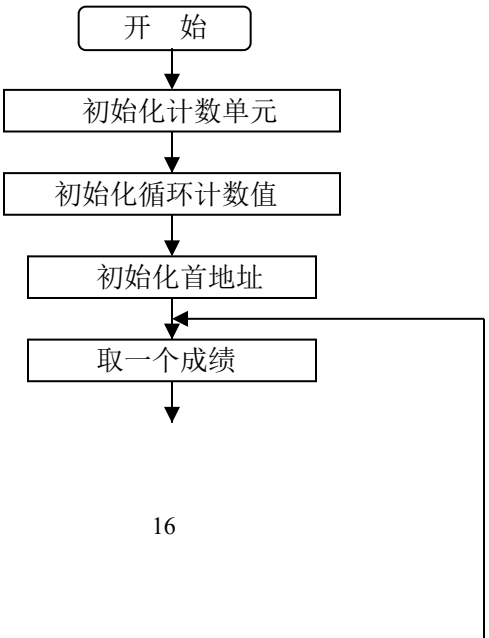
进一步掌握分支程序和循环程序的编写方法。

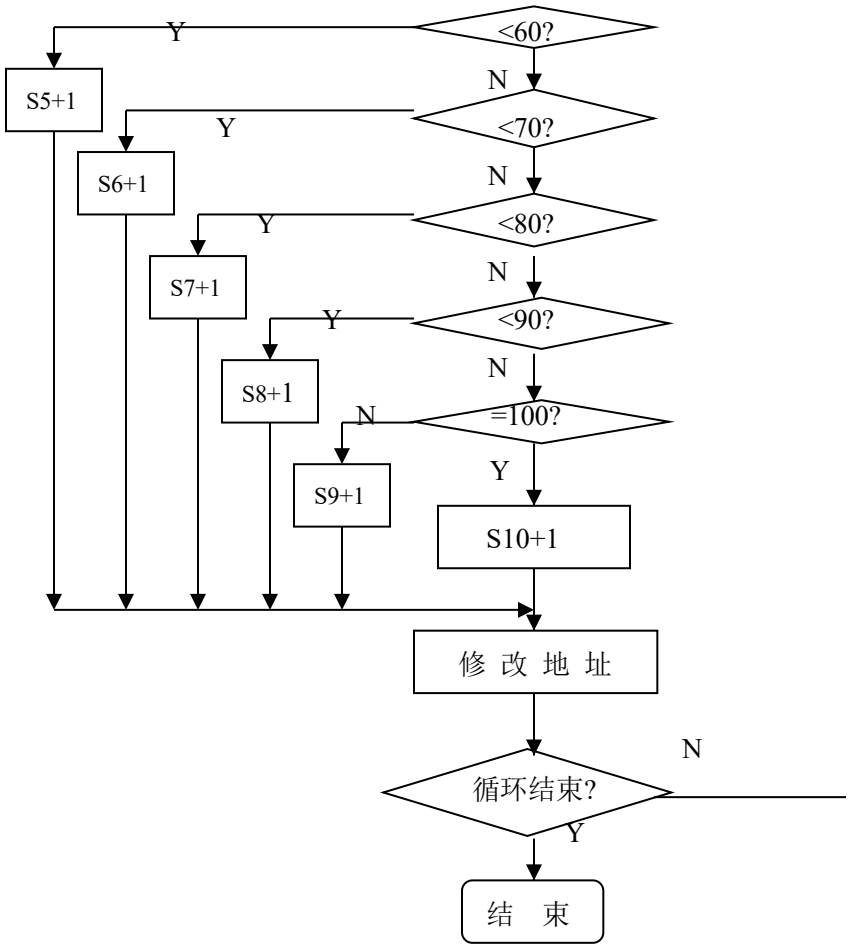
二、实验内容

设有 10 个学生的成绩分别为 56、69、84、82、73、88、99、63、100 和 80 分。试编制程序分别统计低于 60 分、60～69 分、70～79 分、80～89 分、90～99 分及 100 分的人数存放到 s5、s6、s7、s8、s9 及 s10 单元中。

这一题目的算法很简单，成绩分等部分采用分支结构，统计所有成绩则用循环结构完成。程序框图如下图所示。

三、实验步骤





程 序 框 图

1．参考流程图编写实验程序，进行调试。

源代码：

```
DATAS SEGMENT
    GRADE DB 56,69,84,82,73,88,99,63,100,80
    S5 DB 0
    S6 DB 0
    S7 DB 0
    S8 DB 0
    S9 DB 0
```

```
    S10 DB 0
DATAS ENDS

CODES SEGMENT
    ASSUME CS:CODES,DS:DATAS
START:
    MOV AX,DATAS
    MOV DS,AX
    MOV CX,10
    XOR SI,SI
LOOP1:
    MOV AL,GRADE[SI]
    CMP AL,60
    JB LOOP2
    CMP AL,70
    JB LOOP3
    CMP AL,80
    JB LOOP4
    CMP AL,90
    JB LOOP5
    CMP AL,100
    JB LOOP6
    JE LOOP7
LOOP2:
    INC S5
    JMP NEXT
LOOP3:
    INC S6
    JMP NEXT
```

```
LOOP4:
        INC S7
        JMP NEXT
LOOP5:
        INC S8
        JMP NEXT
LOOP6:
        INC S9
        JMP NEXT
LOOP7:
        INC S10
NEXT:
        INC SI
        LOOP LOOP1
        CALL RESULTS

        MOV AH,4CH
        INT 21H

RESULTS PROC
        MOV DL,'S'
        MOV AH,02H
        INT 21H
        MOV DL,'5'
        INT 21H
        MOV DL,':'
        INT 21H
        MOV AL,S5
        CALL NUMBER
```

```
CALL NEWLINE

MOV DL,'S'
INT 21H
MOV DL,'6'
INT 21H
MOV DL,':'
INT 21H
MOV AL,S6
CALL NUMBER
CALL NEWLINE

MOV DL,'S'
INT 21H
MOV DL,'7'
INT 21H
MOV DL,':'
INT 21H
MOV AL,S7
CALL NUMBER
CALL NEWLINE

MOV DL,'S'
INT 21H
MOV DL,'8'
INT 21H
MOV DL,':'
INT 21H
MOV AL,S8
```

```
        CALL NUMBER
        CALL NEWLINE

        MOV DL,'S'
        INT 21H
        MOV DL,'9'
        INT 21H
        MOV DL,':'
        INT 21H
        MOV AL,S9
        CALL NUMBER
        CALL NEWLINE

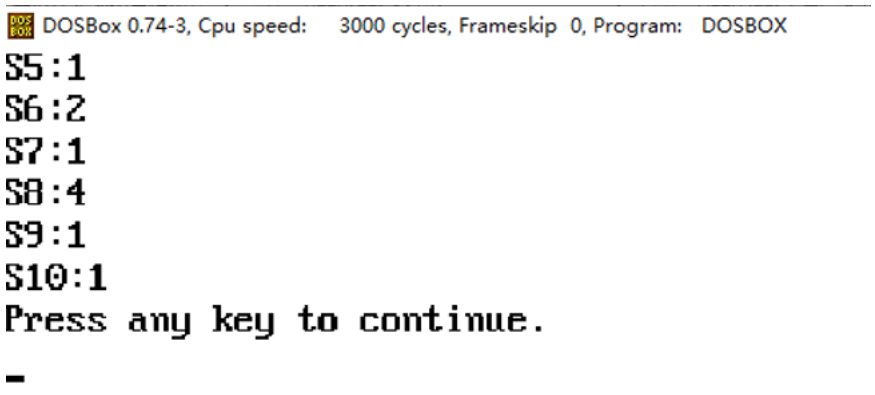
        MOV DL,'S'
        INT 21H
        MOV DL,'1'
        INT 21H
        MOV DL,'0'
        INT 21H
        MOV DL,':'
        INT 21H
        MOV AL,S10
        CALL NUMBER
        CALL NEWLINE
        RET
RESULTS ENDP
NUMBER PROC
        ADD AL,30H
        MOV DL,AL
```

```
    MOV AH,02H
    INT 21H
    RET
NUMBER ENDP

NEWLINE PROC
    MOV DL,0DH
    MOV AH,02H
    INT 21H
    MOV DL,0AH
    MOV AH,02H
    INT 21H
    RET
NEWLINE ENDP

CODES ENDS
    END START
```

2．记录实验结果。



实验六 8254 定时/计数器应用实验

一、 实验目的

1. 掌握 8254 的工作方式及应用编程；
2. 掌握 8254 的典型应用电路接法；
3. 学习 8254 在 PC 系统中的典型应用方法；

二、 实验原理

8254 是可编程间隔定时器。为 8253 的改进型，比 8253 具有更优良的性能。8254-2 具有一些基本功能：

（1）有三个独立的 16 位计数器；

（2）每个计数器可按二进制或十进制计数；

（3）每个计数器可编程工作于 6 种不同工作方式；

（4）8254-2 每个计数器允许的最高计数频率为 10MHz；

（5）8254 有读回命令，除了可以读出当前计数单元的内容外，还可以读出状态寄存器的内容；

（6）计数脉冲可以是有规律的时钟信号，也可以是随机信号。计数初值公式如下，其中 f_{CLKi} 是输入时钟脉冲的频率，f_{OUTi} 是输出脉冲的频率。

$$n=f_{CLKi} / f_{OUTi}$$

图 6-1 是 8254 的内部结构图，它是由与 CPU 的接口、内部控制电路和三个计数器组成。8254 的工作方式如下：

（1） 方式 0：计数到 0 结束输出正跃变信号方式。

（2） 方式 1：硬件可重触发单稳方式。

（3） 方式 2：频率发生器方式。

（4） 方式 3：方波发生器。

（5） 方式 4：软件触发选通方式。

（6） 方式 5：硬件触发选通方式。

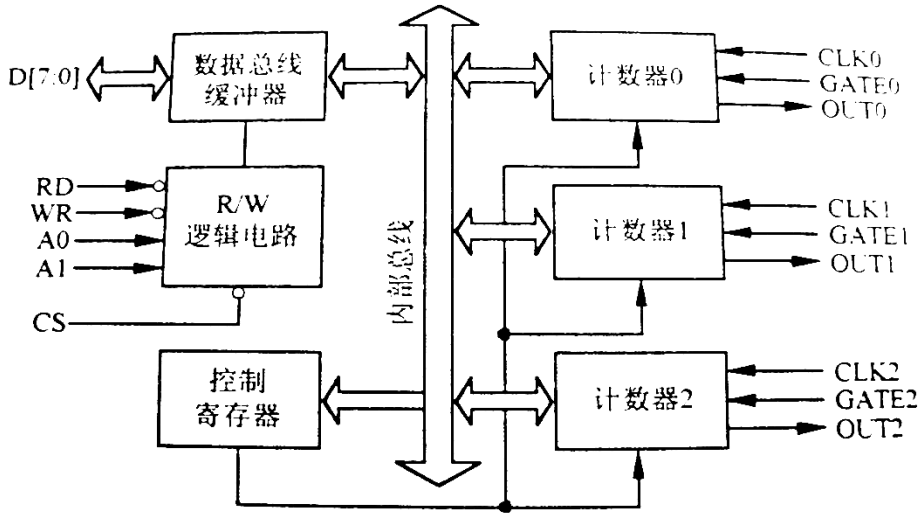


图 6-1

8254 的控制字有两个：一个用来设置计数器的工作方式，称为方式控制字；另一个用来设置读回命令，称为读回控制字。这两个控制字共用一个地址，由标识位来区分。控制字格式如下表所示。

8254 的方式控制字格式

D7	D6	D5	D4	D3	D2	D1	D0
计数器选择		读写格式选择		工作方式选择			计数码制选择
00- 计数器 0 01- 计数器 1 10- 计数器 2 11-读出控制字标志		00-锁存计数值 01-读/写低八位 10-读/写高八位 11-先读/写低八位、再读/写高八位		000-方式 0 001-方式 1 010-方式 2 011-方式 3 100-方式 4 101-方式 5			0-二进制数 1-十进制数

8254 读出控制字格式

OUT 引脚现行状态 1-高电平 0-低电平	计数初值是否装入 1-无效计数 0-计数有效	计数器方式（同方式控制字）

8254 状态字格式

D7	D6	D5	D4	D3	D2	D1	D0
1	1	0-锁存计数值	0-锁存状态信息	计数器选择（同方式控制字）			0

三、 实验设备

微机一台、TD-PITE 实验装置一套。

四、 实验内容

计数应用实验。编写程序应用 8254 的计数功能，将 8254 的计数器 0 设置为方式 0，计数值为十进制 4，用单次脉冲 KK1+作为 CLK0 时钟，OUT0 连接 MIR7，,每当按动 KK1+按动五次后，产生一次计数中断，并在屏幕上显示一个字符“M”。

五、 实验步骤

（1） 按照实验线路图 6-2 连接实验电路

（2） 编写实验程序，经编译、链接无误后装入系统。

（3） 运行程序，按动 KK1+产生单次脉冲，观察实验现象

（4） 改变计数初值，验证 8254 的计数功能。

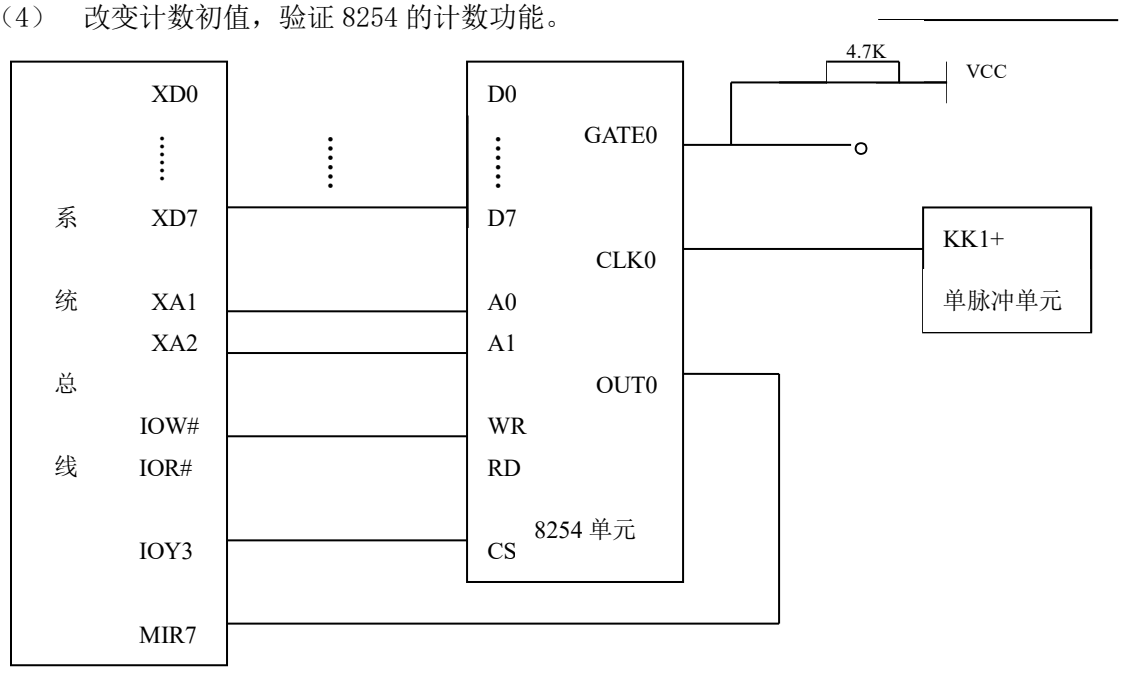


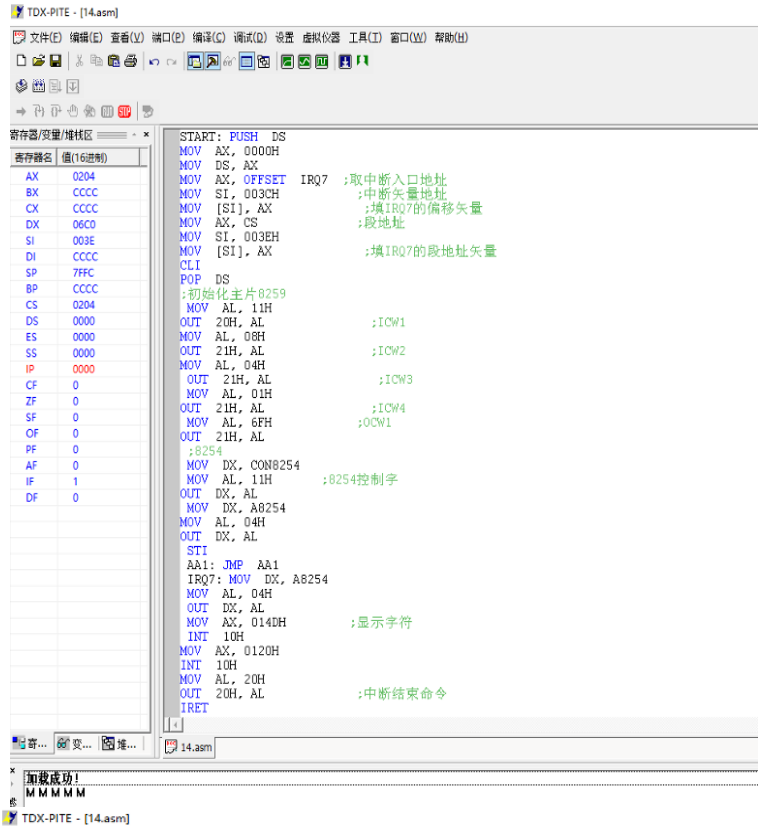
图 6-2 实验接线图

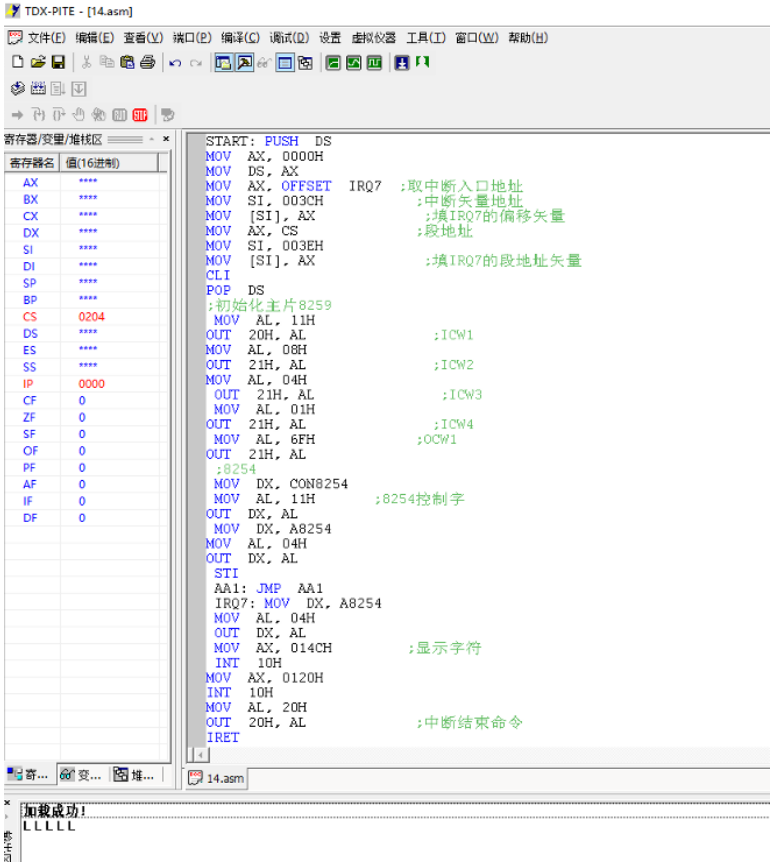
实验程序：

```
A8254   EQU   06C0H
B8254   EQU   06C2H
C8254   EQU   06C4H
CON8254   EQU   06C6H
SSTACK   SEGMENT
STACK   DW   32   DUP(?)
SSTACK   ENDS
CODE   SEGMENT
        ASSUME   CS:CODE,   SS:SSTACK
START: PUSH   DS
         MOV   AX, 0000H
         MOV   DS, AX
         MOV   AX, OFFSET   IRQ7   ;取中断入口地址
         MOV   SI, 003CH           ;中断矢量地址
         MOV   [SI], AX            ;填 IRQ7 的偏移矢量
         MOV   AX, CS              ;段地址
         MOV   SI, 003EH
         MOV   [SI], AX            ;填 IRQ7 的段地址矢量
         CLI
         POP   DS
        ;初始化主片 8259
        MOV   AL, 11H
        OUT   20H, AL              ;ICW1
        MOV   AL, 08H
        OUT   21H, AL              ;ICW2
        MOV   AL, 04H
        OUT   21H, AL              ;ICW3
        MOV   AL, 01H
```

```
        OUT    21H, AL                    ;ICW4
        MOV    AL, 6FH                    ;OCW1
        OUT    21H, AL
        ;8254
        MOV    DX, CON8254
        MOV    AL, 11H            ;8254 控制字
        OUT    DX, AL
        MOV    DX, A8254
        MOV    AL, 04H
        OUT    DX, AL
        STI
 AA1: JMP    AA1
 IRQ7: MOV    DX, A8254
        MOV    AL, 04H
        OUT    DX, AL
        MOV    AX, 014DH              ;显示字符
        INT    10H
        MOV    AX, 0120H
        INT    10H
        MOV    AL, 20H
        OUT    20H, AL                    ;中断结束命令
        IRET
CODE ENDS
        END    START
```

运行截图：


TDX-PITE - [14.asm]
文件(F) 编辑(E) 查看(V) 端口(P) 编译(C) 调试(D) 设置 虚拟仪器 工具(T) 窗口(W) 帮助(H)
寄存器/变量/堆栈区
寄存器名 值(16进制)
AX 0204
BX CCCC
CX CCCC
DX 06C0
SI 003E
DI CCCC
SP 7FFC
BP CCCC
CS 0204
DS 0000
ES 0000
SS 0000
IP 0000
CF 0
ZF 0
SF 0
OF 0
PF 0
AF 0
IF 1
DF 0
START: PUSH DS
MOV AX, 0000H
MOV DS, AX
MOV AX, OFFSET IRQ7 ;取中断入口地址
MOV SI, 003CH ;中断矢量地址
MOV [SI], AX ;填IRQ7的偏移矢量
MOV AX, CS ;段地址
MOV SI, 003EH
MOV [SI], AX ;填IRQ7的段地址矢量
CLI
POP DS
;初始化主片8259
MOV AL, 11H
OUT 20H, AL ;ICW1
MOV AL, 08H
OUT 21H, AL ;ICW2
MOV AL, 04H
OUT 21H, AL ;ICW3
MOV AL, 01H
OUT 21H, AL ;ICW4
MOV AL, 6FH ;OCW1
OUT 21H, AL
;8254
MOV DX, CON8254
MOV AL, 11H ;8254控制字
OUT DX, AL
MOV DX, A8254
MOV AL, 04H
OUT DX, AL
STI
AA1: JMP AA1
IRQ7: MOV DX, A8254
MOV AL, 04H
OUT DX, AL
MOV AX, 014DH ;显示字符
INT 10H
MOV AX, 0120H
INT 10H
MOV AL, 20H
OUT 20H, AL ;中断结束命令
IRET
14.asm
加载成功!
MMMMM


TDX-PITE - [14.asm]
文件(F) 编辑(E) 查看(V) 端口(P) 编译(C) 调试(D) 设置 虚拟仪器 工具(T) 窗口(W) 帮助(H)
寄存器/变量/堆栈区
寄存器名 值(16进制)
AX ****
BX ****
CX ****
DX ****
SI ****
DI ****
SP ****
BP ****
CS 0204
DS ****
ES ****
SS ****
IP 0000
CF 0
ZF 0
SF 0
OF 0
PF 0
AF 0
IF 0
DF 0
START: PUSH DS
MOV AX, 0000H
MOV DS, AX
MOV AX, OFFSET IRQ7 ;取中断入口地址
MOV SI, 003CH ;中断矢量地址
MOV [SI], AX ;填IRQ7的偏移矢量
MOV AX, CS ;段地址
MOV SI, 003EH
MOV [SI], AX ;填IRQ7的段地址矢量
CLI
POP DS
;初始化主片8259
MOV AL, 11H
OUT 20H, AL ;ICW1
MOV AL, 08H
OUT 21H, AL ;ICW2
MOV AL, 04H
OUT 21H, AL ;ICW3
MOV AL, 01H
OUT 21H, AL ;ICW4
MOV AL, 6FH ;OCW1
OUT 21H, AL
;8254
MOV DX, CON8254
MOV AL, 11H ;8254控制字
OUT DX, AL
MOV DX, A8254
MOV AL, 04H
OUT DX, AL
STI
AA1: JMP AA1
IRQ7: MOV DX, A8254
MOV AL, 04H
OUT DX, AL
MOV AX, 014CH ;显示字符
INT 10H
MOV AX, 0120H
INT 10H
MOV AL, 20H
OUT 20H, AL ;中断结束命令
IRET
14.asm
加载成功!
LLLLL

实验七、八　　8255 并行接口

一、实验目的

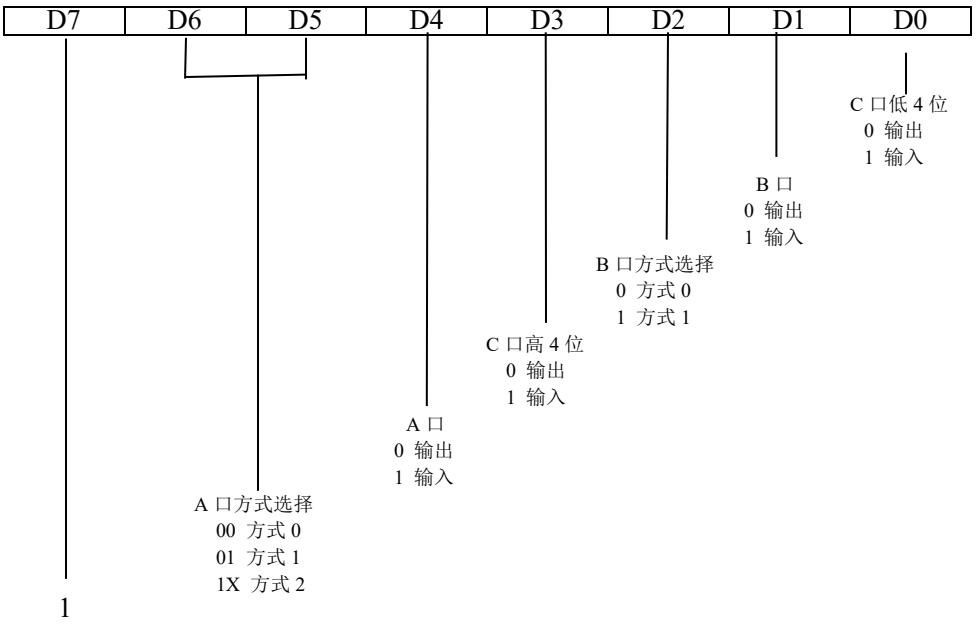
1、 掌握 8255 的工作方式及应用编程。

2、 掌握 8255 的典型应用电路解法。

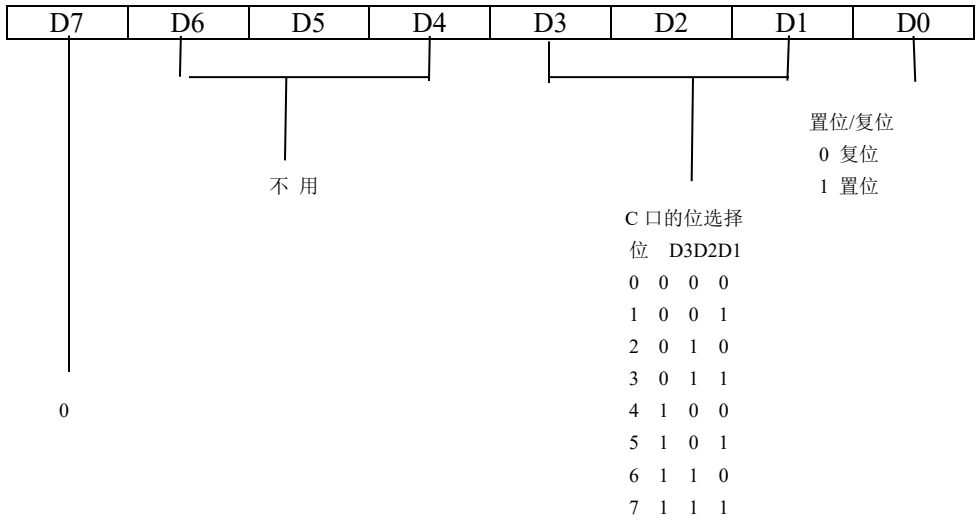
二、实验原理

并行接口是以数据的字节为单位与 I/O 设备或被控制对象之间传递信息。CPU 与接口之间的数据传送总是并行的，即可以同时传递 8 位、16 位、32 位等。

8255 可编程外围接口芯片是通用并行接口芯片，它具有 A、B、C 三个并行接口，用 +5V 单电源供电，能在以下三种方式下工作：方式 0——基本输入/输出方式、方式 1——选通输入/输出方式、方式 2——双向选通工作方式，8255 工作方式控制字和 C 口按位置位/复位控制字格式如图所示。10010000



（a）　工作方式控制字



（b）C 口按位置位/复位控制字

三、实验设备

PC 微机一台，TD-PITE 实验装置一套。

四、实验内容

1、基本输入输出实验。编写程序，使 8255 的 A 口为输入口，端口 B 作为输出口。完成拨动开关到数据灯的数据传输。要求只要开关拨动，数据灯的显示就改变。通过对 8255 芯片编程来实现输入输出功能。

2、流水灯显示实验。编写程序，使 8255 的 A 口和 B 口均为输出，数据灯 D7～D0 由左向右，每次仅亮一个灯，循环显示，D15～D8 与 D7～D0 正相反，由右向左，每次仅点亮一个灯，循环显示。

五、实验步骤

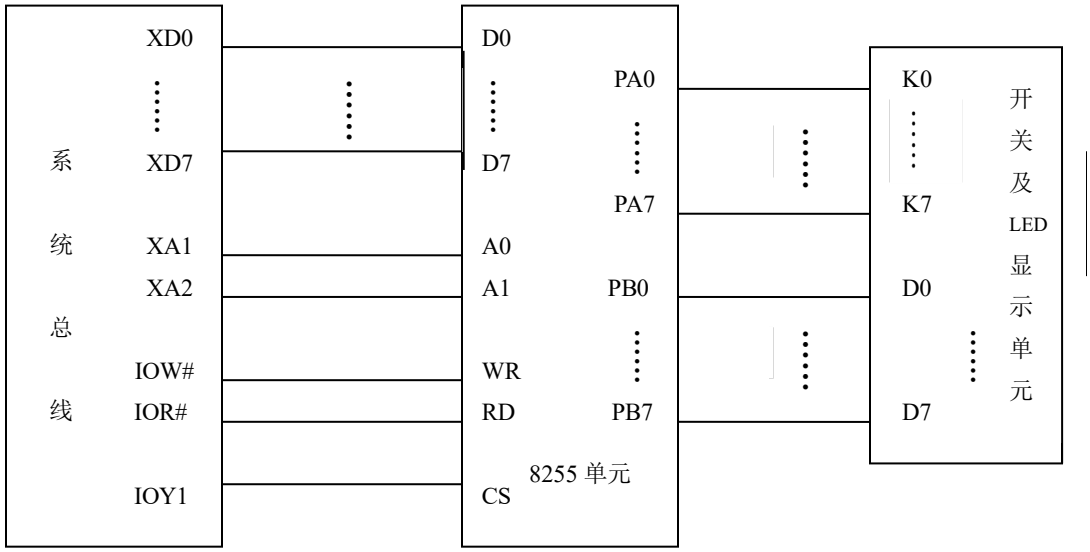
1、基本输入输出实验。

本实验使 8255 端口 A 工作在方式 0 并作为输入口，端口 B 工作在方式 0 并作为输出口。用一组开关信号接入端口 A，端口 B 输出线接至一组数据灯上，然后通过对 8255 芯片编程来实现输入输出功能，扩展 I/O 接口信号线 IOY1 编址空间 0640H~067FH。具体实验步骤如下述：

(1) 按如下实验线路图连接电路。

（2）编写程序，检查无误后汇编、连接。

（3）运行程序，记录拨动开关组时数据灯的显示。



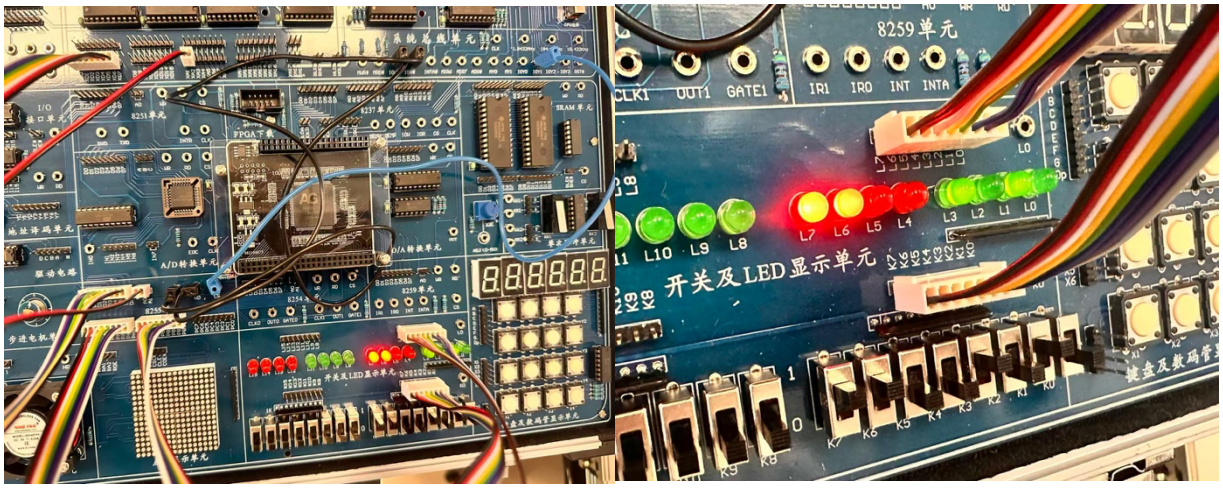
8255 基本输入输出实验接线图

实验程序：

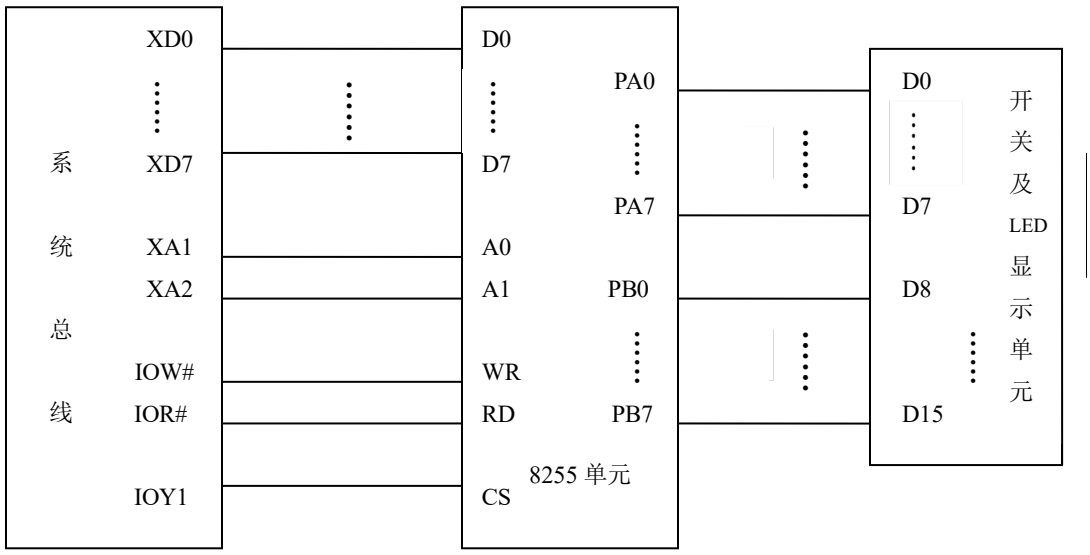
```
SSTACK    SEGMENT
STACK    DW    32    DUP(?)
SSTACK    ENDS
CODE    SEGMENT
ASSUME    CS:CODE
START: MOV    DX, 0646H
          MOV    AL, 90H              ;控制字
          OUT    DX, AL
 AA1:    MOV    DX, 0640H
          IN    AL, DX
          CALL    DELAY
          MOV    DX, 0642H
           OUT    DX, AL
           JMP    AA1
```

```
DELAY: PUSH    CX
            MOV    CX, 0F00H
AA2:      PUSH    AX
            POP    AX
            LOOP    AA2
            POP    CX
            RET
CODE    ENDS
            END    START
```

实验效果图：



8255 的 A 口和 B 口均为输出，，数据灯 D7～D0 由左向右，每次仅亮一个灯，循环显示，D15～D8 与 D7～D0 正相反，由右向左，每次仅点亮一个灯，循环显示。



8255 流水灯实验接线图

运行程序：

```
CODE SEGMENT
    ASSUME CS:CODE
START:
    MOV DX, 0646H
    MOV AL, 80H
    OUT DX, AL
MAIN_LOOP:
    MOV AL, 80H
    MOV AH, 01H
    MOV CX, 8
A_LOOP:
    MOV DX, 0640H
    OUT DX, AL
    MOV DX, 0642H
    MOV DL, 42H
```

```
        XCHG AL, AH
        OUT DX, AL
        XCHG AL, AH
        CALL SHORT_DELAY
        SHR AL, 1
        SHL AH, 1
        LOOP A_LOOP

        JMP MAIN_LOOP

SHORT_DELAY PROC NEAR
        PUSH CX
        MOV CX, 30000
DELAY_LOOP:
        LOOP DELAY_LOOP
        POP CX
        RET
SHORT_DELAY ENDP
CODE ENDS
END START
```

实验效果图：

